

SCR2043 OPERATING SYSTEMS

Name : muhammad mukhritz al iman

Student ID : A23CS0250

Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or Mac OS) environment:

<pre>wget -O mainprocess.c https://rebrand.ly/mainprocess_c wget -O subprocess1.c https://rebrand.ly/subprocess1_c wget -O subprocess2.c https://rebrand.ly/subprocess2_c</pre>

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

<pre>gcc mainprocess.c -o mainprocess gcc subprocess1.c -o subprocess1 gcc subprocess2.c -o subprocess2</pre>

3. Execute the dummy processes:

Run all the dummy processes

<pre>./mainprocess &</pre>

Press enter two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and

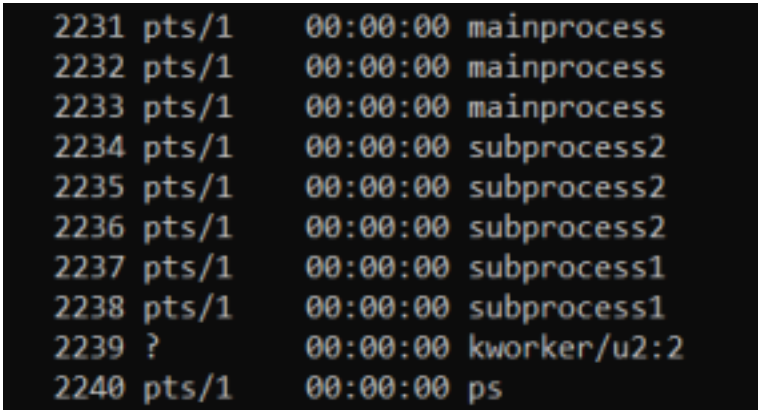
Management Instructions:

1. Carefully execute each command as instructed in the questions.

2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

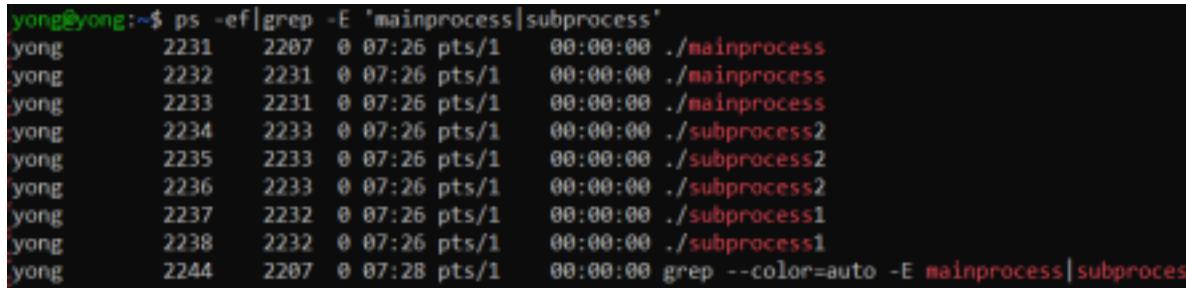
Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command	
<code>ps -e</code>	
Output	
 <pre> 2231 pts/1 00:00:00 mainprocess 2232 pts/1 00:00:00 mainprocess 2233 pts/1 00:00:00 mainprocess 2234 pts/1 00:00:00 subprocess2 2235 pts/1 00:00:00 subprocess2 2236 pts/1 00:00:00 subprocess2 2237 pts/1 00:00:00 subprocess1 2238 pts/1 00:00:00 subprocess1 2239 ? 00:00:00 kworker/u2:2 2240 pts/1 00:00:00 ps </pre>	

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command	
<code>ps -ef grep -E 'mainprocess subprocess'</code>	
Output	
 <pre> yong@yong:~\$ ps -ef grep -E 'mainprocess subprocess' yong 2231 2207 0 07:26 pts/1 00:00:00 ./mainprocess yong 2232 2231 0 07:26 pts/1 00:00:00 ./mainprocess yong 2233 2231 0 07:26 pts/1 00:00:00 ./mainprocess yong 2234 2233 0 07:26 pts/1 00:00:00 ./subprocess2 yong 2235 2233 0 07:26 pts/1 00:00:00 ./subprocess2 yong 2236 2233 0 07:26 pts/1 00:00:00 ./subprocess2 yong 2237 2232 0 07:26 pts/1 00:00:00 ./subprocess1 yong 2238 2232 0 07:26 pts/1 00:00:00 ./subprocess1 yong 2244 2207 0 07:28 pts/1 00:00:00 grep --color=auto -E mainprocess subprocess </pre>	

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command

<code>ps -ef grep -E 'subprocess'</code>
Output
<pre>yong@yong:~\$ ps -ef grep -E 'subprocess' yong 2234 2233 0 07:26 pts/1 00:00:00 ./subprocess2 yong 2235 2233 0 07:26 pts/1 00:00:00 ./subprocess2 yong 2236 2233 0 07:26 pts/1 00:00:00 ./subprocess2 yong 2237 2232 0 07:26 pts/1 00:00:00 ./subprocess1 yong 2238 2232 0 07:26 pts/1 00:00:00 ./subprocess1 yong 2246 2207 0 07:28 pts/1 00:00:00 grep --color=auto -E subprocess</pre>

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
<code>ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess'</code>
Output
<pre>yong@yong:~\$ ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess' 2234 yong 0.0 0.0 ./subprocess2 2235 yong 0.0 0.0 ./subprocess2 2236 yong 0.0 0.0 ./subprocess2 2237 yong 0.0 0.0 ./subprocess1 2238 yong 0.0 0.0 ./subprocess1 2248 yong 0.0 0.1 grep --color=auto -E subprocess</pre>

Question 5

Building on the `ps` command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd -sort=pmem grep -E 'subprocess'</code>
Output

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>sudo renice -5 2093</code>
Output
<pre>yong@yong:~\$ ps -o pid,nice,comm -C subprocess1 PID NI COMMAND 2237 0 subprocess1 2238 0 subprocess1 yong@yong:~\$ sudo renice -5 2237 [sudo] password for yong: 2237 (process ID) old priority 0, new priority -5</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command
<code>killall -15 mainprocess</code>
Output
<pre>yong@yong:~\$ killall -15 mainprocess Main process (ID: 2231) received signal: 15. Terminating... Main process (ID: 2232) received signal: 15. Terminating... Main process (ID: 2233) received signal: 15. Terminating... [1]+ Done ./mainprocess</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
nano example.c
gcc example.c -o example
gcc ./example
```

example.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void child_process() {
    printf("Child process with PID: %d\n", getpid());
    int sleep_time = rand() % 5 + 1;
    printf("Child process sleeping for %d seconds\n", sleep_time);
    sleep(sleep_time);
    printf("Child process exiting\n");
}

int main() {
    printf("Parent process with PID: %d\n", getpid());

    // Fork a child process
    pid_t pid = fork();

    if (pid == 0) {
        // This is the child process
        child_process();
        exit(0);
    } else if (pid > 0) {
        // This is the parent process
        printf("Parent process waiting for child process to finish\n");
        // Wait for the child process to finish
        wait(NULL);
        printf("Parent process exiting\n");
    } else {
        // Error occurred while forking
        perror("fork");
        return 1;
    }

    return 0;
}
```

Output:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void child_process() {
    printf("Child process with PID: %d\n", getpid());
    int sleep_time = rand() % 5 + 1;
    printf("Child process sleeping for %d seconds\n", sleep_time);
    sleep(sleep_time);
    printf("Child process exiting\n");
}

int main() {
    printf("Parent process with PID: %d\n", getpid());

    // Fork a child process
    pid_t pid = fork();

    if (pid == 0) {
        // This is the child process
        child_process();
        exit(0);
    } else if (pid > 0) {
        // This is the parent process

        printf("Parent process waiting for child process to finish\n");
        // Wait for the child process to finish
        wait(NULL);
        printf("Parent process exiting\n");
    } else {
        // Error occurred while forking
        perror("fork");
        return 1;
    }

    return 0;
}

```

Output of the example.c

```

yong@yong:~$ nano example.c
yong@yong:~$ gcc example.c -o example
yong@yong:~$ ./example
Parent process with PID: 2268
Parent process waiting for child process to finish
Child process with PID: 2269
Child process sleeping for 4 seconds
Child process exiting
Parent process exiting

```